Knowledge Graph Report

Knowledge Graph Report

Introduction

This project develops a knowledge graph for London public transportation, centred on the Transport for London (TfL) network and covering lines, stations, zones, operators, accessibility, fares, and night services. The chosen domain matches the coursework brief closely and supports a varied set of competency questions spanning operational, accessibility, and fare-related knowledge.

The repository implements an automated multi-stage construction pipeline rather than relying on manual graph authoring. The current build is orchestrated by `main.py`, which generates the ontology schema through `generateOntology.py`, enriches it with GTFS-aligned modelling, ingests structured TfL API data, processes unstructured Wikipedia text, and merges the outputs into `ontologies/pipeline_output/final_london_transport_kg.ttl`. The current final graph contains 26,156 triples and is approximately 1.1 MB on disk.

This automated design aligns well with the brief's emphasis on combining structured and textual sources, ontology reuse, mapping pipelines, and AI-assisted extraction. The current revision is especially strong in three areas:

- consistent use of the canonical `http://example.org/tfl#` namespace across most generated entities
- programmatic ontology generation and cache-backed reproducibility
- richer provenance and broader source coverage than the earlier snapshot

The public repository linked to this submission is:

- sadiqkhanfanclub

Data Source Selection

Structured sources

The main structured source is the TfL Unified API, cached locally as `downloads/tfl_api_cache.json`. This is the most appropriate structured source in the repository because it provides authoritative operational data directly from the transport authority, including:

- line names and identifiers
- stop-point metadata
- zone information
- service-status information
- selected facility and accessibility attributes

The committed cache is around 79 MB and includes 695 line records together with 7,949 nested stop-point entries. In the current pipeline, `data_retrieval/structured_retrieval/tfl_api_processor.py`

reads this cache directly when present, which makes the committed data both the main structured source and the most reproducible input for marking.

As a supplementary structured resource, the repository also contains `downloads/naptan_london.csv`, which is useful for stop and station normalization. In addition, the historical files `docs/generated/tfl-sources.json` and `docs/generated/tfl-sources.md` remain useful as evidence of earlier source discovery and selection work, even though the present pipeline revision no longer regenerates them directly.

Textual sources

The unstructured stage draws primarily on Wikipedia pages selected in `data_retrieval/unstructured_retri`. The current source list contains 66 article entries, of which 59 are unique. The coverage is intentionally broad so that the graph can capture facts that are difficult to obtain reliably from operational APIs alone, such as:

- line and station descriptions
- operator and service background
- night-service information
- payment-system and fare context
- accessibility-related descriptions
- infrastructure and interchange details

The article inventory includes pages for major Underground lines, interchange stations such as King's Cross St Pancras and Stratford, night-service topics, Oyster and Travelcard concepts, bus operators, and infrastructure pages. This is a good fit for the coursework because it complements the structured TfL data with narrative facts that support competency questions on history, operation, accessibility, and payment.

Source-selection rationale

The overall source strategy follows a sound knowledge-engineering pattern:

1. The TfL API provides authoritative operational facts and identifiers.
2. Wikipedia provides broader contextual facts that are useful for ontology population and competency-question coverage.
3. Local caches preserve reproducibility and reduce dependence on live external services during repeated runs.

This combination is well aligned with the brief because it demonstrates genuine multi-source construction rather than a graph built from a single dataset.

Reproducibility notes

The current repository is designed to run reproducibly from committed caches, while still supporting a configurable live extraction path when needed. The main practical points are:

1. The structured pipeline uses the committed TfL cache by default.
2. The unstructured pipeline preserves a large extraction cache in `data/caches/triples_cache.json`.
3. The live LLM extraction path can be configured locally when a fresh extraction run is required.
4. The source inventory remains explicit in code, making it easy to inspect and refine.

Extension Of Existing Ontologies

The knowledge graph reuses and extends two established ontologies:

- GTFS (`http://vocab.gtfs.org/terms#`)
- Schema.org (`http://schema.org/`)

This is a strong modelling choice because both vocabularies are already well suited to transport, journeys, agencies, stations, and pricing concepts. The current ontology is built programmatically by `generateOntology.py`, which gives the project a reproducible TBox and a clear record of all classes and properties added by the team.

Examples of subclass extensions

The current ontology includes multiple subclasses that extend imported concepts, for example:

- `tfl:TfLLine rdfs:subClassOf gtfs:Route`
- `tfl:TfLStation rdfs:subClassOf gtfs:Station`
- `tfl:OysterFare rdfs:subClassOf schema:PriceSpecification`
- `tfl:TfLOperator rdfs:subClassOf gtfs:Agency`

The ontology then refines these further with domain-specific subclasses such as:

- `tfl:UndergroundLine rdfs:subClassOf tfl:TfLLine`
- `tfl:NightTubeLine rdfs:subClassOf tfl:UndergroundLine`
- `tfl:NightBusRoute rdfs:subClassOf tfl:BusRoute`
- `tfl:UndergroundStation rdfs:subClassOf tfl:TfLStation`
- `tfl:PeakFare rdfs:subClassOf tfl:OysterFare`

These extensions are closely tied to the coursework domain and make the graph much more expressive than imported ontologies alone.

Examples of subproperty extensions

The ontology also defines subproperties that align the local model with established vocabularies, for example:

- `tfl:servedByLine rdfs:subPropertyOf gtfs:route`
- `tfl:operatesInZone rdfs:subPropertyOf gtfs:zone`
- `tfl:operatedBy rdfs:subPropertyOf schema:provider`
- `tfl:fareAmount rdfs:subPropertyOf schema:price`
- `tfl:routeNumber rdfs:subPropertyOf gtfs:shortName`

These choices are important because they preserve compatibility with external ontologies while still allowing the project to express transport-specific semantics such as station facilities, accessibility features, disruptions, journey legs, and fare zones.

Benefits of the current ontology approach

The programmatic generator provides several advantages:

- the ontology can be rebuilt deterministically
- domain additions are visible in a single implementation file
- imported ontologies are declared explicitly
- the class and property hierarchy remains easy to inspect and extend

Overall, the ontology work satisfies the brief well by combining reuse with meaningful domain-specific extension.

Mappings

Structured mapping pipeline

The structured mapping stage is implemented primarily in `data_retrieval/structured_retrieval/tfl_api_`
It reads the TfL cache, normalizes identifiers into the `tfl:` namespace, and emits graph facts from API fields and stop metadata. The current structured stage contributes:

1. line entities and labels
2. station and stop related facts
3. zone assertions
4. accessibility flags
5. facility assertions such as toilets and car parks
6. service-status and disruption-related information where available
7. night-bus classification based on route naming conventions

This stage is a strong part of the current architecture because it grounds the graph in authoritative operational data and produces a large share of the graph's directly verifiable transport facts.

Unstructured mapping pipeline

The unstructured stage combines web-scraped text, chunking, prompt-based triple extraction, and RDF materialization. The main workflow is:

1. select domain-relevant Wikipedia pages in `wiki_scraper.py`
2. split long text into chunks with `CHUNK_SIZE = 1000`
3. extract candidate triples with the constrained prompt in `extract_triples.py`
4. reuse cached extractions where available
5. filter low-value outputs with `remove_junk(...)`
6. convert validated triples into RDF in `triples_to_rdf.py`
7. add enrichment links such as inverse connections and normalized zones

This is a good example of AI-assisted knowledge engineering because the prompt is not used in isolation. Instead, it is embedded inside a broader pipeline with filtering, normalization, and ontology-aware RDF construction.

The current extraction cache contains 1,442 entries, which shows that the project has already accumulated substantial prompt-driven extraction work. The repository snapshot is configured for cache-backed reproducibility, while the live extraction path can be enabled locally when needed.

Merge and build stage

The final build is assembled in `main.py`. The process starts from the generated ontology schema, then merges outputs from structured and unstructured stages into a single final graph. This is an important strength of the current design because it demonstrates an end-to-end construction system rather than isolated scripts.

The merged artifact also includes substantial provenance coverage, with 1,268 subjects linked to explicit `dcterms:source` values and 50 distinct Wikipedia source URIs represented in the final graph. This strengthens the transparency of the knowledge graph and supports downstream evaluation.

Queries

The repository includes 20 competency questions together with an executable validation harness in `competency_questions/competency_question_test.py`. This is valuable because it ties the graph directly to the project requirements and provides a concrete way to measure whether the knowledge graph answers the intended questions.

On the current final graph, the stored SPARQL harness returns non-empty results for 12 out of the 20 competency questions. This gives a current non-empty answerability score of:

- 12/20 answered
- 60% non-empty query coverage

The present query set is strongest on operational and service-oriented questions. Representative examples include:

- CQ1: King's Cross St Pancras returns a clear set of intersecting Underground lines
- CQ4: DLR returns zone information
- CQ5: the Jubilee line query returns step-free-access-related station results
- CQ17: night bus routes return a substantial set of `N`-prefixed services

This means the graph is already supporting a meaningful part of the intended question set through executable SPARQL queries. The remaining questions provide a focused target list for the separate completion-analysis work required by the brief.

Evaluation Methodology

Quality metrics

The evaluation strategy focuses on metrics that are appropriate for an automated coursework knowledge graph:

1. competency-question answerability through executable SPARQL queries
2. qualitative review of returned answers for relevance and precision
3. graph coverage metrics such as number of triples and populated domain areas
4. provenance coverage
5. ontology-conformance checks during inspection of generated entities and predicates

Useful current graph indicators include:

- final merged graph size: 26,156 triples
- subjects with explicit `dcterms:source`: 1,268
- distinct Wikipedia source URIs represented in the graph: 50
- night bus routes: 75
- public toilets: 43
- car parks: 27
- assisted boarding features: 2
- fare concessions: 5

These figures show broad automated coverage across several important parts of the domain, especially services, facilities, and provenance.

Performance metrics

A fresh local measurement on the current final graph gives:

- parse time: about 1.26 seconds
- total 20-query evaluation time: about 0.29 seconds

- average per-query time: about 0.015 seconds
- peak resident memory during that run: about 61 MB

These results indicate that the graph is lightweight enough to load and query efficiently in a coursework setting, while still being large enough to demonstrate a substantial automated build.

Evaluation considerations

The current repository supports reliable evaluation because:

1. the final graph is committed and directly testable
2. the SPARQL competency-question harness is executable
3. key data sources are cached locally
4. prompts and mapping code are preserved in the repository

As with any evolving automated pipeline, some evaluation dimensions benefit from iterative refinement, particularly for the subset of questions that depend on narrowly targeted benchmark instances. Those areas are documented in the completion analysis and in the separate engineering note.

Conclusion

The current repository demonstrates a credible automated knowledge-graph construction system for the London public-transport domain. It combines structured TfL data, unstructured textual extraction, ontology reuse, domain-specific extensions, provenance capture, and executable SPARQL evaluation in a single reproducible workflow.

The strongest aspects of the project are its clear multi-source design, programmatic ontology generation, cache-backed reproducibility, and measurable support for a substantial portion of the competency-question set. Taken together, these features align well with the brief and provide a solid basis for the completion and prompt-analysis documents that accompany the report.